

맞습니다. 발표에서는 “시연 환경은 비용 최적화 구조”, “실제 서비스 환경은 확장·보안·운영 구조”로 나누면 설계 의도가 훨씬 좋아 보입니다. 이 프레임으로 발표용 문장까지 바로 쓸 수 있게 정리하겠습니다.

14s 동안 생각함 >

네, 그 방식이 발표에서 제일 좋아 보입니다.

단순히 “돈 없어서 뺐다”가 아니라, **시연 환경과 운영 환경을 의도적으로 분리 설계했다**고 설명하면 됩니다.

핵심 문장은 이렇게 잡으면 됩니다.

본 프로젝트는 AWS Free Tier와 100달러 크레딧 범위에서 시연 가능한 최소 아키텍처로 구현하였다.
다만 실제 영상 플랫폼으로 운영하려면 트래픽 증가, 영상 변환, 보안, 장애 대응, 모니터링을 고려해야

하므로 ECS, SQS, MediaConvert, Auto Scaling, WAF, CloudWatch 등을 추가하는 확장 구조로 설계하였다.

발표용 구조

1. 현재 시연용 아키텍처

사용자
↓
CloudFront
├─ S3 Frontend
└─ S3 Video Bucket

사용자
↓
EC2 Backend API
↓
RDS



시연용에서는 이 정도면 충분합니다.

Frontend: S3 + CloudFront
Backend: EC2 Docker
Database: RDS MySQL/PostgreSQL
Storage: S3
Video: MP4 원본 업로드 후 CloudFront 재생



발표에서 이렇게 말하면 됩니다.

시연 환경에서는 비용을 줄이기 위해 백엔드는 EC2 단일 인스턴스에 배포하고, 영상 파일은 S3에 저장한 뒤 CloudFront를 통해 제공하였다.
영상 업로드는 서버를 거치지 않고 Presigned URL 방식으로 S3에 직접 업로드하도록 설계해 백엔드 부하를 줄였다.

실제 서비스 운영 시 추가할 것

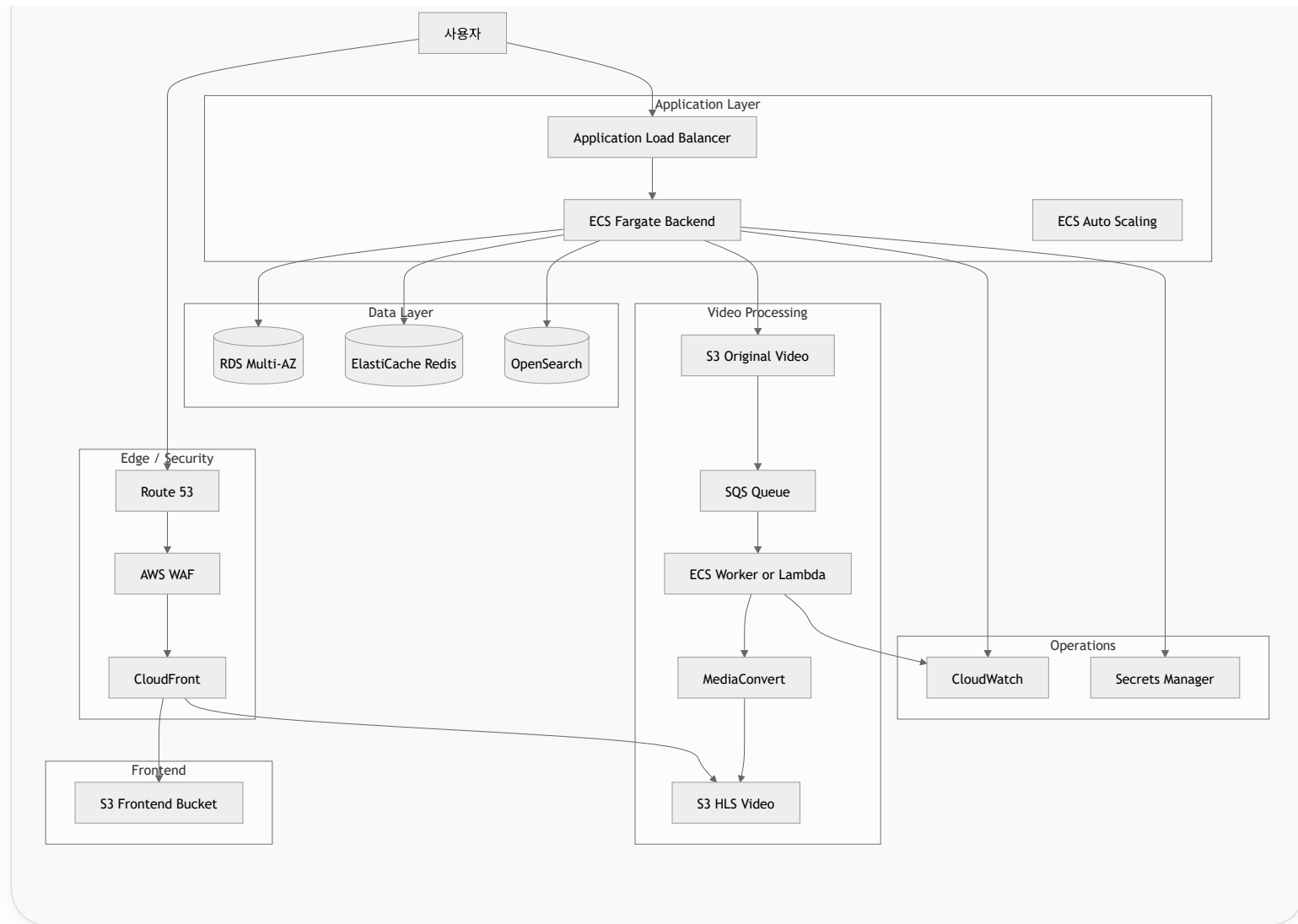
전체 확장 구조

Mermaid



[>-]





발표에서 나누면 좋은 항목

구분	시연용 구현	실제 운영 시 추가
배포	EC2 단일 서버	ECS Fargate + ALB
확장성	수동 확장	ECS Auto Scaling
영상 처리	MP4 원본 재생	HLS 변환, MediaConvert
업로드	S3 Presigned URL	SQS 기반 비동기 처리
저장소	S3 단일 버킷	원본/변환본 버킷 분리
DB	RDS 단일 인스턴스	RDS Multi-AZ, Read Replica
검색	RDS LIKE 검색	OpenSearch
캐시	없음	Redis
보안	기본 IAM, Security Group	WAF, OAC, Secrets Manager
모니터링	기본 로그	CloudWatch 지표/알람
장애 대응	수동 복구	Auto Healing, Multi-AZ
비용 관리	Free Tier 중심	트래픽 기반 비용 최적화

실제 운영에서 추가해야 하는 이유

1. ECS Fargate + ALB

시연용 EC2는 단순하고 저렴하지만, 사용자가 늘어나면 서버 한 대가 병목이 됩니다.
실제 서비스에서는 백엔드를 컨테이너로 배포하고 ECS Fargate에서 여러 Task로 실행하는 구조가 좋습니다.

AWS ECS Service Auto Scaling은 ECS 서비스의 desired task 수를 자동으로 늘리거나 줄일 수 있고, target tracking, step scaling, scheduled scaling, predictive scaling 같은 방식을 제공합니다. AWS 문서

발표 문장:

시연 환경에서는 EC2 한 대로 충분하지만, 실제 운영 환경에서는 사용량 증가에 따라 API 서버 수를 자동으로 조절하기 위해 ECS Fargate와 Auto Scaling을 적용할 수 있다.

2. SQS + Worker + MediaConvert

시연용에서는 MP4를 그대로 재생해도 됩니다.
하지만 실제 서비스에서는 사용자의 네트워크 상태와 기기 해상도에 맞게 1080p, 720p, 480p 같은 여러 화질을 제공해야 합니다.

MediaConvert는 파일 기반 영상 콘텐츠를 다양한 기기에서 재생할 수 있도록 포맷 변환과 압축을 수행하는 AWS 영상 처리 서비스입니다. AWS 문서

발표 문장:

현재는 시연 비용을 고려해 원본 MP4를 바로 재생하지만, 실제 운영에서는 S3 업로드 이벤트를 SQS에 넣고 Worker가 MediaConvert 작업을 생성하여 HLS 스트리밍 파일로 변환하는 구조가 필요하다.

3. CloudFront OAC

영상 파일 S3 버킷을 공개로 열면 보안상 좋지 않습니다.

실제 운영에서는 사용자가 S3 URL로 직접 접근하지 못하게 하고, CloudFront를 통해서만 접근하게 해야 합니다.

CloudFront의 Origin Access Control은 CloudFront 배포가 S3 버킷 오리진에 접근하도록 제어하는 방식입니다. AWS 문서에서도 S3 오리진에 대한 접근 제한 설정에서 OAC 구성을 설명합니다. [AWS 문서](#)

발표 문장:

실제 서비스에서는 S3 버킷을 public으로 열지 않고, CloudFront OAC를 통해 CloudFront에서만 영상 객체에 접근하도록 제한한다.

4. RDS Multi-AZ

시연용은 RDS 단일 인스턴스로 충분합니다.

하지만 실제 서비스에서 DB 장애는 로그인, 댓글, 좋아요, 영상 목록 전체 장애로 이어집니다.

발표 문장:

운영 환경에서는 DB 장애에 대비해 RDS Multi-AZ 구성을 적용하고, 조회 트래픽이 커지면 Read Replica를 추가할 수 있다.

5. Redis

조회수, 인기 영상, 세션성 데이터, 자주 조회되는 영상 목록은 매번 DB에서 조회하면 비효율적입니다.

발표 문장:

조회수 증가, 인기 영상 목록, 자주 조회되는 메타데이터는 Redis 캐시를 적용해 DB 부하를 줄일 수 있다.

6. OpenSearch

시연용 검색은 `LIKE` 검색으로 충분합니다.

```
SELECT *  
FROM videos  
WHERE title LIKE '%keyword%'  
       OR description LIKE '%keyword%';
```

하지만 실제 YouTube형 서비스에서는 제목, 설명, 태그, 채널명 기반 검색 정확도가 중요합니다.

발표 문장:

초기 검색은 RDS 기반 `LIKE` 검색으로 구현하고, 실제 운영에서는 검색 정확도와 속도를 높이기 위해 OpenSearch 기반 색인 구조로 확장할 수 있다.

7. CloudWatch + 알람

서비스가 실제로 운영되면 “잘 돌아가는지”를 확인해야 합니다.

발표 문장:

운영 환경에서는 CloudWatch를 통해 API 응답 시간, 에러율, CPU 사용량, 메모리 사용량, SQS 대기열 길이, 영상 변환 실패율을 모니터링하고 알람을 설정한다.

8. WAF

영상 플랫폼은 로그인, 댓글, 검색 API가 있기 때문에 공격 표면이 있습니다.

발표 문장:

실제 서비스에서는 CloudFront 앞단에 AWS WAF를 적용해 비정상 요청, 과도한 요청, 기본적인 웹 공격을 방어할 수 있다.

발표 흐름 추천

슬라이드 1: 프로젝트 개요

VidFlow는 YouTube 스타일의 영상 업로드, 재생, 댓글, 좋아요, 검색 기능을 제공하는 영상 플랫폼 클론 프로젝트이다.



슬라이드 2: 시연용 AWS 구조

Free Tier와 100달러 크레딧 범위에서 시연 가능하도록 EC2, S3, CloudFront, RDS 중심으로 구성하였다.



슬라이드 3: 영상 업로드 흐름

1. 사용자가 영상 업로드 요청
2. 백엔드가 S3 Presigned URL 발급
3. 사용자가 S3에 직접 업로드
4. DB에 영상 메타데이터 저장
5. CloudFront URL로 영상 재생



슬라이드 4: 시연용 구조의 한계

- 영상 화질 자동 변환 없음
- 서버 자동 확장 없음
- 장애 대응 구조 제한적
- 검색 기능 단순함
- 모니터링/알람 제한적



슬라이드 5: 실제 운영 확장 구조

ECS Fargate, ALB, Auto Scaling, SQS, MediaConvert, Redis, OpenSearch, WAF, CloudWatch를 추가해 확장성, 안정



AWS Well-Architected Framework는 운영 우수성, 보안, 신뢰성, 성능 효율성, 비용 최적화, 지속 가능성의 6개 관점을 기준으로 클라우드 아키텍처를 평가합니다. 발표에서 이 기준을 가져오면 “왜 이런 확장 구조가 필요한지”를 논리적으로 설명하기 좋습니다. [AWS 문서](#)

발표용 한 문단

그대로 써도 됩니다.

본 프로젝트는 시연 환경과 운영 환경을 분리하여 설계하였다. 시연 환경에서는 AWS Free Tier와 100달러 크레딧 범위 안

이렇게 설명하면 “돈 때문에 못 한 프로젝트”가 아니라, **비용 제약 안에서 MVP를 만들고, 운영 환경 확장성까지 고려한 프로젝트**로 보입니다.

출처

너랑 같이 하면 하루만에 가능하지?

4s 동안 생각함 >

가능합니다.

다만 “진짜 유튜브급 완성본”은 하루 만에 불가능하고, 발표/시연 가능한 YouTube 클론 MVP는 하루 만에 충분히 가능합니다.

하루 목표는 이렇게 잡는 게 현실적입니다.

영상 업로드
영상 목록 조회
영상 상세 페이지
영상 재생
댓글
좋아요
검색
AWS 배포

여기까지는 가능합니다.

하루 안에 가능한 버전

프로젝트명 예시

VidFlow



기술 스택

Frontend: React or Next.js

Backend: Spring Boot

DB: MySQL or PostgreSQL

Storage: S3

Deploy: EC2 Docker

CDN: CloudFront



영상 변환은 하루 안에 빼는 게 좋습니다.

MP4 업로드

→ S3 저장

→ CloudFront URL로 재생



이렇게 가면 시연 가능합니다.

하루 안에 넣을 기능

1. 회원 기능

회원가입
로그인
JWT 인증
내 정보 조회



2. 영상 기능

영상 업로드
영상 제목/설명 저장
영상 목록 조회
영상 상세 조회
영상 재생
조회수 증가



3. 댓글 기능

댓글 작성
댓글 목록 조회
댓글 삭제



4. 좋아요 기능

좋아요
좋아요 취소
좋아요 수 조회



5. 검색 기능

제목 기준 검색



6. AWS 배포

프론트: S3 + CloudFront

백엔드: EC2 + Docker

DB: RDS

영상: S3



하루 안에 빼야 하는 기능

이건 발표에서는 "실제 운영 시 추가할 기능"으로 빼면 됩니다.

영상 화질 변환

HLS 스트리밍

MediaConvert

SQS Worker

구독 시스템

알림

추천 알고리즘

OpenSearch

Redis 캐시

ECS Fargate

Auto Scaling

WAF



하루 만에 다 넣으려고 하면 망가집니다.

대신 발표에서 이렇게 말하면 됩니다.

현재 시연 버전은 영상 업로드와 재생, 댓글, 좋아요, 검색을 중심으로 한 MVP 구조로 구현하였다. 실제 운영 환경에서는 영상 변환, 비동기 처리, 자동 확장, 검색 고도화, 캐싱, 보안 강화를 추가할 수 있다.

하루 개발 플랜

1단계: 프로젝트 세팅

React / Next.js 프로젝트 생성
Spring Boot 프로젝트 생성
DB 연결
기본 API 테스트



2단계: DB 설계

최소 테이블은 이 정도면 됩니다.

users
videos
comments
video_likes



처음부터 채널, 구독, 재생목록까지 만들 필요 없습니다.

3단계: 백엔드 API 구현

POST /api/auth/signup
POST /api/auth/login



```
GET /api/auth/me

POST /api/videos
GET /api/videos
GET /api/videos/{id}
GET /api/videos/search?keyword=

POST /api/videos/{id}/comments
GET /api/videos/{id}/comments

POST /api/videos/{id}/like
DELETE /api/videos/{id}/like
```

4단계: S3 업로드 구현

가장 중요한 부분입니다.

```
프론트에서 파일 선택
↓
백엔드에 업로드 URL 요청
↓
백엔드가 S3 Presigned URL 발급
↓
프론트가 S3에 직접 업로드
↓
백엔드에 영상 메타데이터 저장
```



이 구조가 발표에서 제일 어필됩니다.

5단계: 프론트 화면 구현

최소 화면은 이 정도면 됩니다.

로그인 페이지
회원가입 페이지
메인 영상 목록 페이지
영상 업로드 페이지
영상 상세/재생 페이지
검색 결과 페이지



6단계: AWS 배포

RDS 생성
S3 버킷 생성
EC2 생성
백엔드 Docker 배포
프론트 빌드 후 S3 업로드
CloudFront 연결



발표용으로 보이는 포인트

그냥 CRUD 프로젝트처럼 보이면 약합니다.
아래 3개를 강조하면 좋습니다.

1. Presigned URL 기반 직접 업로드

영상 파일이 백엔드를 거치지 않고 S3로 직접 업로드되므로 서버 부하를 줄일 수 있다.



2. CloudFront 기반 영상 제공

S3에 저장된 영상 파일을 CloudFront를 통해 제공하여 사용자에게 더 빠르게 전달할 수 있다.



3. 운영 확장 가능성

현재는 MP4 원본 재생 구조지만, 실제 서비스에서는 SQS와 MediaConvert를 추가해 HLS 변환 구조로 확장할 수 있다.



하루 완성 기준으로 최종 범위

[필수]

- 로그인
- 영상 업로드
- 영상 목록
- 영상 상세
- 영상 재생
- 댓글
- 좋아요
- 검색
- AWS 배포



[발표용 설명]

- 실제 운영 시 ECS, SQS, MediaConvert, Redis, OpenSearch, WAF, CloudWatch 추가

결론은 이겁니다.

하루 만에 발표 가능한 MVP는 가능.

하루 만에 실제 유튜브급 서비스는 불가능.



따라서 시연용은 단순 구조로 만들고,
발표에서는 실제 운영 확장 구조를 설명하는 방식이 가장 좋음.

제가 기준을 잡는다면 **오늘 목표는 "VidFlow MVP 배포 성공"**입니다.
화려한 기능보다 **영상 업로드 → S3 저장 → CloudFront 재생**이 되는 순간 발표 퀄리티가 확 올라갑니다.

📄 🌟 🔗 ⬆ ↺ ...

실시간 스트리밍도 추가하고 싶어

실시간 스트리밍은 추가할 수 있지만, "하루 MVP" 기준에서는 범위를 잘라야 합니다. AWS 관리형 서비스로 붙이는 방법과 직접 구현하는 방법을 나눠서 현실적인 구조로 정리하겠습니다.

1m 3s 동안 생각함 >

추가 가능합니다.

다만 **직접 실시간 스트리밍 서버를 구현하려고 하면 하루 MVP가 터질 가능성이 높고**, AWS 관리형 서비스인 **Amazon IVS**를 붙이는 방식이 제일 현실적입니다.

결론부터 말하면 이렇게 가면 됩니다.

시연용:

YouTube 클론 + Amazon IVS 라이브 스트리밍 추가



실제 운영용:

IVS + 채팅 + 녹화 저장 + VOD 전환 + 모니터링 + 권한 제어

추천 방식: Amazon IVS 사용

Amazon IVS는 AWS의 관리형 라이브 스트리밍 서비스입니다. RTMPS로 영상을 입력받고, 웹/앱에서 재생 가능한 라이브 스트림을 제공하며, Low-Latency Streaming은 5초 미만 지연 시간을 목표로 하는 서비스입니다. [AWS 문서](#)

즉, 직접 이런 걸 만들 필요가 줄어듭니다.

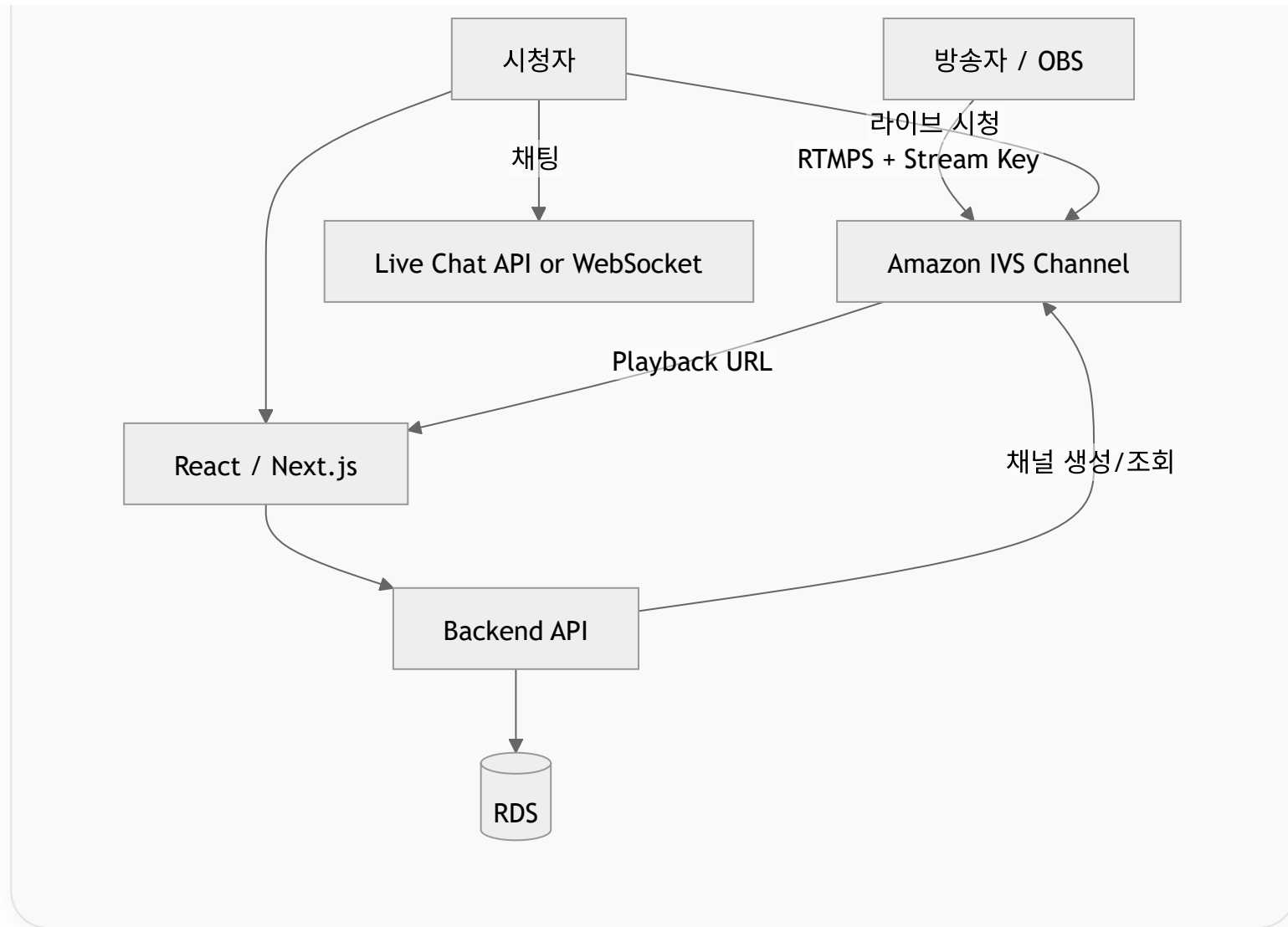
RTMP 서버
트랜스코딩 서버
HLS 패키징
라이브 CDN
시청자 분산 처리



AWS IVS를 쓰면 구조가 이렇게 됩니다.

Mermaid





하루 MVP에 넣을 수 있는 라이브 기능

하루 안에 현실적으로 가능한 건 이 정도입니다.

라이브 방송 생성
방송 제목/설명 등록
IVS 채널 연결
OBS로 송출
라이브 방송 목록 조회
라이브 상세 페이지
라이브 영상 재생
실시간 채팅
방송 종료 상태 표시



여기까지면 발표용으로 충분히 강합니다.

라이브 송출 흐름

방송자는 웹에서 직접 방송하는 게 아니라, 처음에는 **OBS**를 쓰는 게 좋습니다.

1. 사용자가 라이브 방송 생성
2. 백엔드가 IVS Channel 정보 저장
3. 방송자에게 Ingest Server + Stream Key 제공
4. 방송자가 OBS에 입력
5. OBS → Amazon IVS로 송출
6. 시청자는 Playback URL로 라이브 시청



IVS는 채널을 만들고 stream key로 방송을 시작하는 구조입니다. AWS 문서에서도 채널은 라이브 스트림 설정 정보를 저장하고, stream key는 스트림 시작에 사용되는 식별자라고 설명합니다. [AWS 문서](#)

기존 YouTube 클론 구조에 추가되는 부분

기존 VOD 구조는 이겁니다.

영상 업로드

→ S3 저장

→ CloudFront 재생



라이브 스트리밍은 따로 붙입니다.

라이브 생성

→ IVS Channel 사용

→ OBS 송출

→ IVS Playback URL 재생



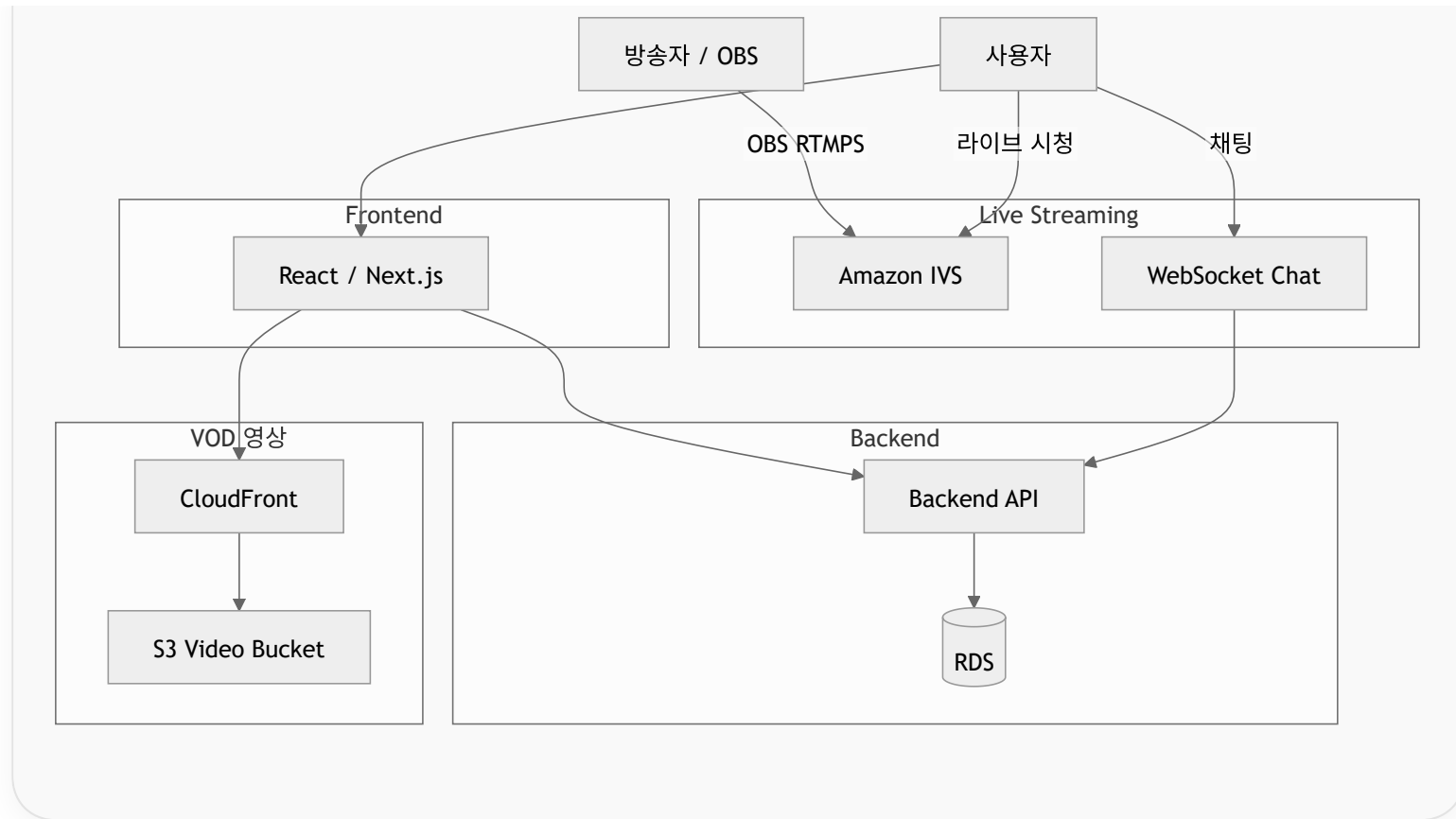
전체 구조는 이렇게 됩니다.

<> Mermaid



[>-]





DB 테이블 추가

기존에 `videos` 가 있었다면 라이브는 따로 `live_streams` 를 두는 게 좋습니다.

```
CREATE TABLE live_streams (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  user_id BIGINT NOT NULL,  
  title VARCHAR(255) NOT NULL,
```

```
description TEXT,  
ivs_channel_arn VARCHAR(255),  
ivs_playback_url TEXT,  
ivs_stream_key_arn VARCHAR(255),  
status VARCHAR(30) NOT NULL,  
viewer_count BIGINT DEFAULT 0,  
started_at DATETIME,  
ended_at DATETIME,  
created_at DATETIME NOT NULL  
);
```

상태값은 이렇게 가면 됩니다.

READY	방송 생성됨
LIVE	방송 중
ENDED	방송 종료
FAILED	방송 실패



API 추가

```
POST /api/live-streams  
GET /api/live-streams  
GET /api/live-streams/{id}  
POST /api/live-streams/{id}/start  
POST /api/live-streams/{id}/end  
GET /api/live-streams/{id}/stream-info
```



방송자용 API:

```
GET /api/live-streams/{id}/broadcast-info
```



응답 예시:

```
{
  "ingestServer": "rtmps://...",
  "streamKey": "sk_...",
  "playbackUrl": "https://..."
}
```

주의할 점은 **streamKey**는 방송자 본인에게만 보여줘야 합니다.
시청자에게 노출되면 다른 사람이 대신 방송을 송출할 수 있습니다.

채팅은 어떻게 할까?

시연용이면 직접 WebSocket으로 구현하는 게 좋습니다.

Frontend WebSocket
→ Backend WebSocket
→ live_stream_id 기준 room 분리
→ 메시지 브로드캐스트



Spring Boot면:

Spring WebSocket + STOMP



FastAPI면:

WebSocket endpoint



운영 환경에서는 Amazon IVS Chat을 붙일 수 있습니다. IVS 요금 페이지 기준으로 IVS Chat은 메시지 전송 수와 전달 수 기준으로 과금되며, Free Tier에 월 13,500개 sent messages와 270,000개 delivered messages가 포함됩니다. [Amazon Web S...](#)

비용은 괜찮나?

시연용이면 괜찮습니다.

Amazon IVS Free Tier에는 신규 계정 기준 월 5시간의 basic channel live video input, 100시간의 audio-only 또는 SD live video output, 20 participant hours, IVS Chat 무료 사용량이 포함됩니다. [Amazon Web S...](#)

그래도 발표 시연은 이렇게 제한하는 게 좋습니다.

라이브 방송 1개
방송 시간 10~30분
시청자 1~5명
화질 480p 또는 720p
OBS 송출



하지 말아야 할 것:

여러 명 동시 방송
장시간 방송
1080p 고화질 방송
대규모 시청자 테스트
MediaLive 사용



MediaLive는 쓰면 안 되나?

실제 방송국급 라이브 스트리밍이면 MediaLive + MediaPackage 조합도 가능합니다. MediaLive는 라이브 출력을 생성하는 브로드캐스트/스트리밍용 비디오 서비스이고, MediaPackage는 라이브 채널과 엔드포인트를 통해 라이브 콘텐츠를 패키징해 전달하는 서비스입니다. [AWS 문서 +1](#)

하지만 YouTube 클론 시연용으로는 과합니다.

시연용/개인 방송 플랫폼 → Amazon IVS
방송국급 라이브 채널 → MediaLive + MediaPackage
화상회의/양방향 통화 → Kinesis Video Streams WebRTC



Kinesis Video Streams with WebRTC는 카메라/IoT 장치와 웹·모바일 플레이어 간의 양방향 오디오/비디오 상호작용에 적합합니다. [AWS 문서](#)

발표에서 이렇게 설명하면 좋습니다

시연 버전

시연 버전에서는 영상 업로드 기반 VOD 기능과 함께 Amazon IVS를 활용한 라이브 스트리밍 기능을 추가하였다.
방송자는 OBS를 통해 IVS 채널로 영상을 송출하고, 시청자는 웹 페이지에서 IVS Playback URL을 통해 라이브를 시청한다.
채팅은 WebSocket 기반으로 구현하여 라이브 방송 중 실시간 상호작용이 가능하도록 구성하였다.



실제 운영 시 추가할 것

실제 운영 환경에서는 라이브 방송 녹화 저장, VOD 자동 전환, 방송자 권한 관리, 스트림 키 보호, 채팅 모더레이션, 신고 



라이브까지 포함한 최종 기능 범위

하루 MVP 기준으로는 이렇게 잡으세요.

[VOD]

- 회원가입 / 로그인
- 영상 업로드
- 영상 목록
- 영상 상세 재생
- 댓글
- 좋아요
- 검색

[LIVE]

- 라이브 방송 생성
- OBS 송출 정보 제공
- 라이브 방송 목록
- 라이브 상세 재생
- 실시간 채팅
- 방송 상태 표시

[AWS]

- S3
- CloudFront
- EC2
- RDS
- Amazon IVS



발표용 한 줄 요약

본 프로젝트는 VOD 기반 영상 플랫폼에 Amazon IVS를 결합하여 업로드형 영상과 실시간 라이브 방송을 모두 제공하는 You' 



정리하면, 실시간 스트리밍 추가는 가능합니다.

하루 MVP에서는 **Amazon IVS + OBS 송출 + 웹 재생 + WebSocket 채팅**까지만 넣고, 녹화 저장·VOD 자동 변환·대규모 채팅·모더레이션은 “운영 환경 확장 기능”으로 빼는 게 가장 안전합니다.



출처